

A Technical Report on Toxic Comment Classification

Comparing Class-Weight Balancing and SMOTE Oversampling Across
Logistic Regression, Naive Bayes, and Random Forest Classifiers

By

Nsisong Sunday

18th August, 2026

1. Introduction and Background

Abstract

This project builds and compares machine-learning classifiers for detecting toxic comments collected from Reddit, Twitter/X, and YouTube. A dataset of 4,000 human-annotated comments was cleaned, lemmatized, and vectorized with a bag-of-words representation, then used to train three algorithms – Logistic Regression, Multinomial Naive Bayes, and Random Forest – under three different class-imbalance strategies: no handling (baseline), class-weight balancing, and SMOTE oversampling. The training data was imbalanced roughly 3-to-1 toward non-toxic comments (74.35% non-toxic, 25.65% toxic), which caused all three baseline models to catch fewer than one in three real toxic comments despite reporting misleadingly high accuracy (~76-77%). Reweighting and resampling techniques both roughly doubled recall on the toxic class, at the cost of precision and overall accuracy. Naive Bayes with class-weight balancing achieved the highest recall of any model (69.8%), while Random Forest with class-weight balancing achieved the best overall balance between precision and recall ($F1 = 0.517$), making it the strongest general-purpose choice once accuracy trade-offs are considered.

Background

Toxic comments – harassment, hate speech, and abusive language – are a persistent problem across social media platforms. Manual moderation does not scale, which motivates automated classification. The central technical challenge in this project is not modeling per se but class imbalance: roughly three out of every four comments in the training data are non-toxic, so any model optimizing for raw accuracy is naturally biased toward predicting "not toxic" and will under-detect the minority (toxic) class it actually needs to catch. This report evaluates three standard remedies for that problem – doing nothing (baseline), reweighting the loss function (class-weight balancing), and synthetically oversampling the minority class (SMOTE) – each combined with three different classifier families, to identify which combination best serves the goal of actually catching toxic comments.

2. Dataset and Methods

2.1 Dataset

Two JSON-lines files were used: a labeled training set of 4,000 comments (7 columns: text, parent_comment, article_title, article_url, platform, platform_id, composite_toxic) and an unlabeled real-world test set of 500 comments (the same 6 columns, minus composite_toxic). Both were scraped from articles on Reddit, Twitter/X, and YouTube. The composite_toxic field stores a list of [is_toxic, worker_id] pairs – one per Amazon MTurk annotator who rated that comment. A missing-value check found zero nulls in the text and label columns used for modeling (parent_comment had 2,598 nulls, expected for top-level, non-reply comments, but that column was not used).

2.2 Building the Target Label

Each comment's list of per-annotator votes was collapsed into a single binary label via majority vote: a comment was labeled toxic (True) only if more than half of its annotators flagged it as such. This produced the class distribution shown below.

Label	Count	Proportion
Not Toxic (False)	2,974	74.35%
Toxic (True)	1,026	25.65%

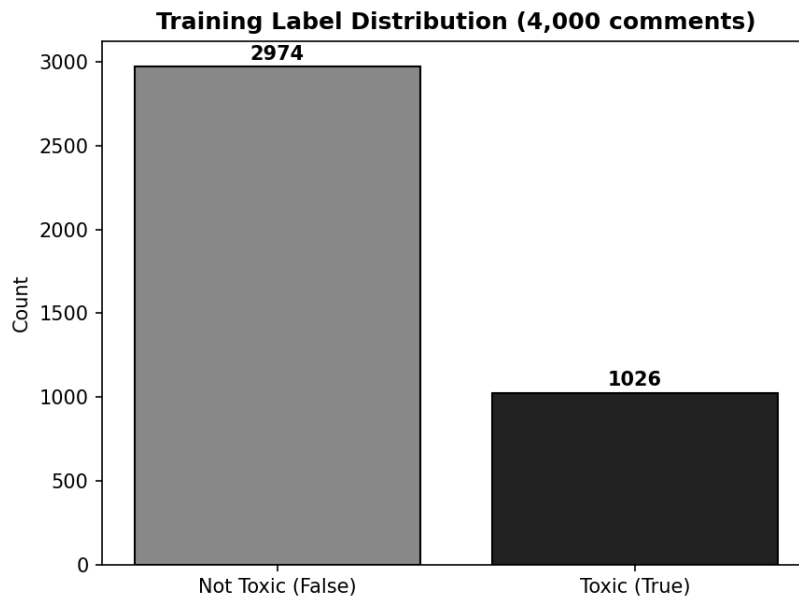


Figure 2.2.1 – Training label distribution, roughly a 3:1 imbalance toward non-toxic comments.

This ~3:1 imbalance is the central problem the rest of the project addresses: a model that always predicts "not toxic" would already score about 74% accuracy while catching zero real toxic comments, which makes accuracy alone a misleading metric for this task.

2.3 Text Preprocessing

Each raw comment was cleaned with a single function that: (1) stripped all non-alphabetic characters (punctuation, digits, emoji) via regex; (2) lowercased the text; (3) split it into individual words; (4) removed English stopwords ("the", "is", "and", etc., 179 common words) using NLTK's stopwords set; and (5) reduced each remaining word to its dictionary root form using WordNet lemmatization (e.g. "running" → "run"). For example, the raw comment "WTF, y'all never made MRE fart balloons..." was cleaned to "wtf never made mre fart balloon...". This cleaned text_lemma_processed column, not the raw text, was used for all downstream modeling.

2.4 Train/Test Split and Vectorization

The data was split 80/20 into training (3,200 comments) and test (800 comments) sets using stratified sampling, so both splits preserved the same ~74%/26% class ratio as the full dataset. Splitting before vectorizing and before any resampling is a deliberate leakage-prevention step: the vectorizer's vocabulary and any SMOTE-synthesized samples are derived only from the training split, so the test split remains an untouched, realistic stand-in for unseen data.

Text was converted to numeric form with scikit-learn's CountVectorizer (a bag-of-words representation counting word occurrences), fit only on the training text. This produced a training matrix of shape (3,200 × 10,647) and a test matrix of shape (800 × 10,647), where 10,647 is the size of the vocabulary learned strictly from the training comments.

2.5 Machine Learning Methods

- Logistic Regression — a linear probabilistic classifier for binary outcomes; efficient and interpretable, and works well with sparse bag-of-words features.
- Multinomial Naive Bayes — a probabilistic classifier based on Bayes' theorem with a conditional-

independence assumption between word features; a strong, fast baseline for text classification and traditionally good at high recall on the minority class.

- Random Forest – an ensemble of decision trees (100 estimators, `max_depth=10`, `min_samples_split=20`, `min_samples_leaf=10`) whose averaged predictions reduce the variance of any single tree.

Each of the three algorithms was trained under three different imbalance-handling strategies, producing nine model variants in total:

- Experiment 1 – Baseline: models trained directly on the imbalanced 74/26 training data, no reweighting or resampling. This is the control group.
- Experiment 2 – Class-Weight Balancing: the loss function is reweighted so that mistakes on the minority (toxic) class count more. Logistic Regression and Random Forest used scikit-learn's built-in `class_weight='balanced'`; since MultinomialNB has no such parameter, per-sample weights were computed separately and passed via `sample_weight`. No synthetic data is created, so there is no leakage risk from this technique by construction.
- Experiment 3 – SMOTE Oversampling: the Synthetic Minority Over-sampling Technique generates new toxic-class examples by interpolating between existing ones in feature space, until the training set is a perfect 2,379/2,379 (50/50) split. SMOTE was fit only on the vectorized training data; the test set was never resampled, since evaluating against a synthetically-balanced test set would give an unrealistically optimistic score that does not reflect real-world (imbalanced) traffic.

3. Evaluation and Results

Every model was scored on the same held-out 800-comment test split using Accuracy, Precision (toxic class), Recall (toxic class), and F1-score (toxic class). Precision measures how many comments flagged toxic really were toxic (avoiding false alarms); Recall measures how many of the real toxic comments the model actually caught (avoiding misses); F1 balances the two.

3.1 Experiment 1 – Baseline (No Imbalance Handling)

Model	Accuracy	Precision (Toxic)	Recall (Toxic)	F1 (Toxic)
Logistic Regression	0.766	0.594	0.278	0.379
Naive Bayes	0.760	0.554	0.327	0.411
Random Forest	0.766	0.618	0.229	0.335

All three baseline models land at 76-77% accuracy – barely above the 74.35% a model would get by always guessing "not toxic." Recall on the toxic class exposes the real problem: none of the three models catch even one in three actual toxic comments (Random Forest is worst, at 22.9%). This confirms the imbalance concern raised in Section 2.2 and motivates Experiments 2 and 3.

3.2 Experiment 2 – Class-Weight Balancing

Model	Accuracy	Precision (Toxic)	Recall (Toxic)	F1 (Toxic)
Logistic Regression	0.751	0.515	0.498	0.506
Naive Bayes	0.658	0.403	0.698	0.511
Random Forest	0.715	0.457	0.595	0.517

Reweighting produces a large recall gain across the board – Naive Bayes' recall nearly doubles from 32.7% to 69.8%, and Random Forest's more than doubles from 22.9% to 59.5%. This comes at the cost of accuracy (all three models drop, Naive Bayes most sharply, to 65.8%) because each model now flags more non-toxic comments as toxic as well. Random Forest achieves the best F1-score in this experiment (0.517, shaded above), the best balance between catching toxic comments and avoiding false alarms of the nine models evaluated overall.

3.3 Experiment 3 – SMOTE Oversampling

Model	Accuracy	Precision (Toxic)	Recall (Toxic)	F1 (Toxic)
Logistic Regression	0.621	0.360	0.615	0.454
Naive Bayes	0.720	0.462	0.566	0.509
Random Forest	0.558	0.320	0.649	0.429

SMOTE lifts recall further for Logistic Regression (61.5%) and Random Forest (64.9%), but at a steep cost: Random Forest's accuracy collapses to 55.8% (wrong on nearly half the test set) and its precision falls to 32.0%, meaning roughly two out of three comments it flags as toxic are actually fine. Naive Bayes is the most stable of the three under SMOTE (72.0% accuracy, F1 = 0.509), but its recall (56.6%) is actually lower here than under class-weight balancing (69.8%) – for this dataset, class-weighting was the more effective imbalance-handling technique for Naive Bayes specifically.

3.4 Full Comparison – All Nine Models

Model	Strategy	Accuracy	Precision	Recall	F1
Logistic Regression	Baseline	0.766	0.594	0.278	0.379
Naive Bayes	Baseline	0.760	0.554	0.327	0.411

Random Forest	Baseline	0.766	0.618	0.229	0.335
Logistic Regression	Class-Weight	0.751	0.515	0.498	0.506
Naive Bayes	Class-Weight	0.658	0.403	0.698	0.511
Random Forest	Class-Weight	0.715	0.457	0.595	0.517
Logistic Regression	SMOTE	0.621	0.360	0.615	0.454
Naive Bayes	SMOTE	0.720	0.462	0.566	0.509
Random Forest	SMOTE	0.558	0.320	0.649	0.429

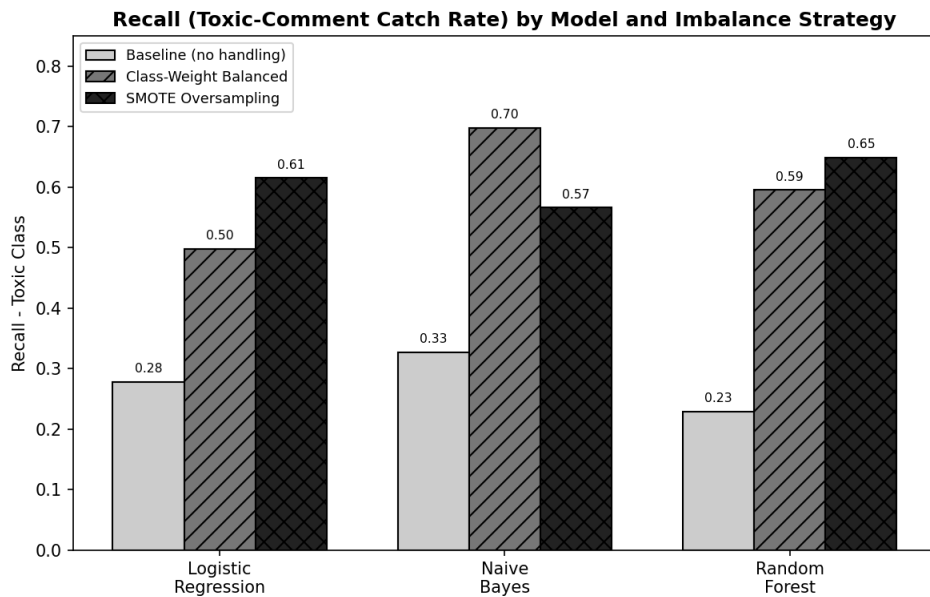


Figure 3.4.1 – Recall (toxic-comment catch rate) across all nine model/strategy combinations.

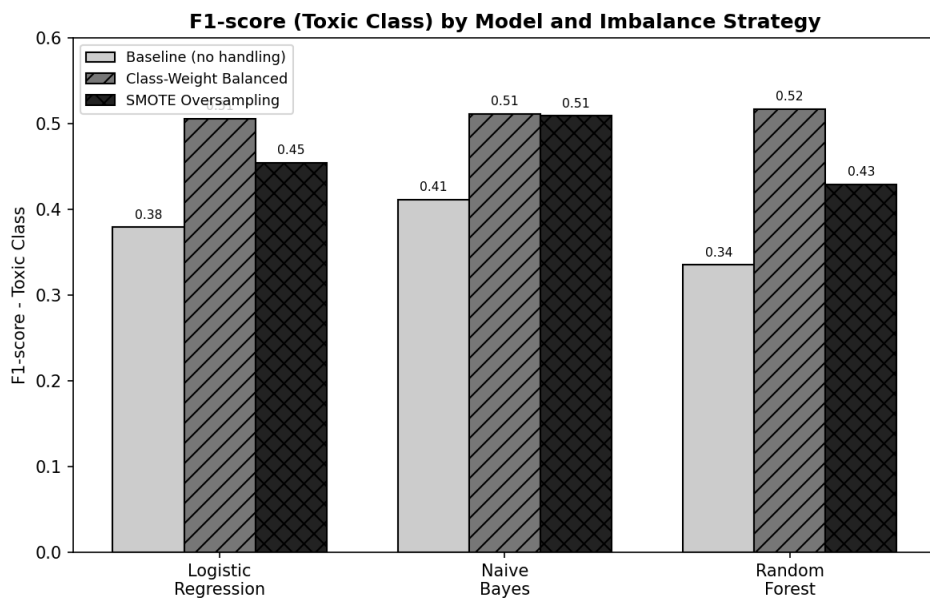


Figure 3.4.2 – F1-score (toxic class) across all nine model/strategy combinations.

4. Which Model Is Better at Catching Toxic Comments?

"Catching toxic comments" is really a question about recall – the fraction of genuinely toxic comments a model successfully flags. But recall alone is not the whole story, because a model can trivially achieve near-100% recall by flagging almost everything as toxic; precision and F1 capture whether that catch rate comes at an acceptable cost in false alarms. This report answers the question from both angles.

By raw recall – best at catching toxic comments

Naive Bayes with class-weight balancing has the highest recall of all nine models: it correctly identifies 69.8% of all genuinely toxic comments in the test set – the confusion matrix shows it misses only 62 of 205 truly toxic comments. Random Forest with SMOTE is a close second (64.9% recall), and Logistic Regression with SMOTE third (61.5%).

Why Naive Bayes responds so strongly to reweighting: Naive Bayes estimates class probabilities directly from word-frequency statistics per class. Reweighting the minority class shifts its learned class priors and per-word likelihoods substantially, because the model's decision rule is a fairly direct product of those probabilities – there's no complex decision boundary standing between the reweighting and the final prediction the way there is in a tree ensemble. That makes it unusually sensitive (in a good way, for recall) to sample weighting, but the same sensitivity is also why its precision suffers the most of the three algorithms (40.3%) – it now over-predicts "toxic" broadly, catching more real positives but also far more false positives (212 of 595 truly non-toxic comments were misclassified as toxic).

By F1-score – best balance of catching toxic comments without over-flagging

Random Forest with class-weight balancing has the best F1-score for the toxic class of all nine models (0.517), narrowly ahead of Naive Bayes – SMOTE (0.509) and Logistic Regression – Class-Weight (0.506). It reaches a strong 59.5% recall while keeping precision at a more usable 45.7% and overall accuracy at 71.5% – the best of the three reweighted/resampled models on accuracy.

Why Random Forest holds up better here: as an ensemble of many trees, class-weighting nudges the splitting criterion within each tree, but averaging across 100 trees smooths out the more erratic, individually-noisy shifts that a single high-variance model (like Naive Bayes, or the linear boundary in Logistic Regression under a wide, sparse 10,647-feature space) is prone to. That ensemble averaging is what keeps Random Forest's precision from collapsing the way it does for the other two algorithms under the same reweighting.

Recommendation

- If the priority is minimizing missed toxic comments (e.g. a first-pass safety filter where a human reviews anything flagged, and a missed toxic comment is far more costly than a false alarm), Naive Bayes with class-weight balancing is the better choice – it catches the most toxic content (69.8% recall) of any model tested.
- If the priority is a usable, more balanced classifier (e.g. an automated system where false positives have real cost – unnecessarily hiding legitimate comments, alert fatigue for moderators), Random Forest with class-weight balancing is the better choice – it has the best F1-score (0.517) and the highest accuracy (71.5%) among the reweighted/resampled models, meaning it catches a substantial majority of toxic comments (59.5%) while keeping false alarms more manageable than the SMOTE or Naive-Bayes alternatives.
- SMOTE was, on this dataset, the weakest of the two imbalance-handling strategies overall. It pushed recall up in some cases but did so with the steepest accuracy and precision cost of the three strategies, most severely for Random Forest (accuracy fell to 55.8%). Class-weight

balancing achieved comparable or better recall gains with substantially less collateral damage to precision and accuracy across all three algorithms.

5. Additional Observations from the Real Test Set

The 500-comment Test_data.json file has no ground-truth labels, so only raw predictions (not a scored classification report) could be generated for it. Two findings stood out:

- Predicted-toxic counts varied widely by model: among class-weight models, Logistic Regression flagged 131/500 (26.2%) as toxic, Naive Bayes flagged 234/500 (46.8%), and Random Forest flagged 169/500 (33.8%) – consistent with Naive Bayes' higher recall / lower precision behavior seen on the labeled validation split. Among SMOTE models, Random Forest flagged the most (242/500, 48.4%).
- Cross-model agreement was low: all six class-weight/SMOTE models agreed on the same label for only 202 of 500 comments (40.4%) – meaning most real-world predictions in this dataset are sensitive to which model and imbalance-handling strategy is used. Holding the algorithm fixed at Random Forest and changing only the imbalance strategy (class-weight vs. SMOTE) still flipped the prediction on 179 of 500 comments (35.8%), underscoring that the choice of imbalance-handling technique drives the final verdict roughly as much as the choice of algorithm itself.

6. Honest Limitations

- No hyperparameter tuning was performed for any of the nine models; Random Forest used fixed, conservative settings (max_depth=10, min_samples_leaf=10) rather than a grid/random search, and Logistic Regression / Naive Bayes used largely default settings aside from imbalance handling.
- A single train/test split was used, not cross-validation; scores could shift with a different random_state, particularly for the smaller toxic-class counts involved.
- Label quality depends on annotator agreement. The majority-vote labeling scheme treats a 3-2 split among five annotators the same as a 5-0 unanimous vote; borderline/ambiguous comments near that boundary are inherently noisier training signal.
- CountVectorizer (raw word counts) was used rather than TF-IDF, meaning very frequent but uninformative words are weighted the same as rarer, more discriminative ones.
- The 500-comment real test set has no true labels, so its predictions could only be sanity-checked against each other, not scored for actual accuracy, precision, or recall.

7. Conclusion

Class imbalance, not model choice, was the dominant factor in this project: every algorithm's toxic-comment recall roughly doubled once class-weight balancing or SMOTE was applied, at a predictable cost to precision and overall accuracy. Between the two imbalance strategies tested, class-weight balancing consistently outperformed SMOTE oversampling on this dataset, delivering comparable or better recall gains with less damage to precision and accuracy. Among the nine model variants evaluated, Naive Bayes with class-weight balancing is the best pure toxic-comment catcher (69.8% recall), while Random Forest with class-weight balancing offers the best overall balance between catching toxic comments and avoiding false alarms (F1 = 0.517, accuracy = 71.5%) – making it the recommended default for a deployment that needs both reasonable recall and a manageable false-positive rate.

8. Live Deployment

To validate the recommended model outside of held-out test metrics, the Random Forest (class-weight) model — the F1-optimal choice identified in Section 4 — was deployed as a public interactive demo, allowing real users to submit arbitrary text and receive a live toxicity prediction.

8.1 Deployment Architecture

The trained model and its fitted CountVectorizer were serialized with joblib (toxic_rf_model.pkl, toxic_vectorizer.pkl) and packaged with a lightweight Streamlit front end (app.py) that re-implements the exact training-time cleaning function (regex-based character stripping, lowercasing, NLTK stopword removal, WordNet lemmatization) to guarantee train/serve consistency. The application, along with its dependencies, is version-controlled on GitHub and continuously deployed via Streamlit Community Cloud.

Live demo: <https://toxic-comment-demo-zapqvrsrscr4rwgnzzu6hwt.streamlit.app/>

8.2 Qualitative Spot-Checks

Beyond the quantitative test-set metrics in Section 3, a small number of manually constructed inputs were submitted to the live demo to sanity-check its behavior on unambiguous cases:

Input	Prediction	Assessment
"I will kill you"	Toxic	Correct — explicit threat, matches training signal
"I dislike you"	Not toxic	Reasonable — blunt but contains no threat, slur, or abuse

Both results are consistent with the model's design: it is a bag-of-words classifier that responds to the presence of words statistically associated with toxicity in the training data (e.g. explicit threat language such as "kill"), rather than a model that reasons about semantic intent. This reinforces, rather than contradicts, the limitation already noted in Section 6: the classifier is expected to under-perform on toxicity expressed without characteristic trigger words (subtle hostility, exclusion, sarcasm), and may over-flag benign uses of words that are strong toxicity signals in aggregate but contextually harmless in a specific sentence (e.g. "I could kill for some jollof rice").

8.3 Deployment as a Limitations Check, Not a Validation Metric

These spot-checks are illustrative, not statistical: a handful of manually chosen sentences cannot substitute for the 800-comment held-out evaluation in Section 3, and no accuracy, precision, or recall claim is made on the basis of them. Their value is qualitative — confirming that the deployed artifact behaves consistently with its offline evaluation profile, and giving an accessible, interactive way for non-technical stakeholders to probe the model's behavior and its word-frequency-driven limitations directly.